

DOCUMENTAZIONE SMART-DRIVE

OOP-BD

1. DESCRIZIONE DEL SISTEMA

Il sistema informativo “Smart drive” è un’applicazione software progettata per amministrare in modo centralizzato il ciclo di vita del noleggio di una flotta di auto. L’applicativo permette al personale autorizzato di monitorare lo stato del parco mezzi, l’anagrafica dei clienti e stipulare contratti di noleggio con questi ultimi.

Il cuore pulsante dell’applicativo risiede nella sua capacità di individuare e gestire automaticamente problemi di integrità (es. validità di documenti di guida, prevenzioni delle sovrapposizioni di date) e di calcolare dinamicamente le tariffe di fatturazione. Queste ultime vengono elaborate incrociando i dati commerciali del contratto (giorni previsti) con i dati logistici reali rilevati al momento della consegna e del rientro del mezzo (chilometraggio effettivo, ritardi, stato di usura).

2. AMBITO DI UTILIZZO (DOMINIO)

Il dominio di utilizzo dell’applicativo è quello di un autonoleggio a sede unica. Il sistema è pensato per essere utilizzato direttamente dagli operatori al banco (desk clerk) e dall’amministrazione.

L’obiettivo aziendale che il software si propone di risolvere è la riduzione degli errori umani nelle fasi di check-out e check-in dei

veicoli. Digitalizzando le "Schede di Stato del Veicolo" e separando concettualmente il contratto di Noleggio dalla sua Fattura, l'azienda ottiene un tracciamento rigoroso e in tempo reale della disponibilità della flotta e dei ricavi generati.

FUNZIONAMENTO LOGICO

Il funzionamento logico del sistema si articola in 4 flussi principali:

□ Controllo anagrafiche e flotta

1. L'operatore potrà inserire i dati anagrafici del cliente, inserendo tutte le informazioni richieste (es. numero di patente, data di scadenza della patente)
2. Viene popolato l'inventario dei Veicoli, distinti per categoria (Auto o Furgone), definendone la tariffa giornaliera e le specifiche tecniche (es. volume di carico, categoria).

□ Apertura del noleggio (check-out)

1. Il cliente richiede un mezzo per un periodo specifico. Il sistema interroga il database per garantire la disponibilità del mezzo richiesto, bloccando eventuali tentativi di overbooking.
2. Al momento della consegna delle chiavi, l'operatore compila una Scheda di Uscita, "congelando" a sistema i chilometri iniziali, il livello di carburante e certificando l'avvenuto lavaggio.

□ Chiusura del noleggio (check-in)

1. Al momento della riconsegna del veicolo l'operatore avrà l'obbligo di compilare la scheda apposita, in cui verranno segnalati eventuali danni e calcolati i chilometri percorsi.
2. Il sistema rileverà automaticamente la data di riconsegna effettiva e calcolerà eventuali ritardi.

□ Fatturazione e contabilità

1. Sulla base dei dati raccolti dalle schede di ispezione e dalle date del contratto, il sistema (tramite la logica applicativa del Controller) genera la Fattura.
2. Viene calcolato l'importo base (giorni pattuiti) e vengono applicate in automatico le penali qualora sussistano ritardi nella consegna o superamenti di soglie chilometriche non pattuite.

NOTE ARCHITETTURALI

Per garantire la robustezza del sistema, le regole di business più critiche sono demandate direttamente alla base di dati PostgreSQL tramite l'utilizzo di trigger (es. blocco dell'inserimento di un noleggio se la patente del cliente risulta scaduta o se i chilometri di rientro sono inferiori a quelli di uscita). L'interazione utente e i calcoli contabili sono invece isolati e gestiti nell'applicativo Java seguendo rigorosamente il design pattern BCE (Boundary-Control-Entity) combinato con DAO per la persistenza dei dati.

Link alla Repository GitHub

<https://github.com/ipug89/Progetto-POO-BD-2025-2026>

DIAGRAMMA DELLE CLASSI (UML)

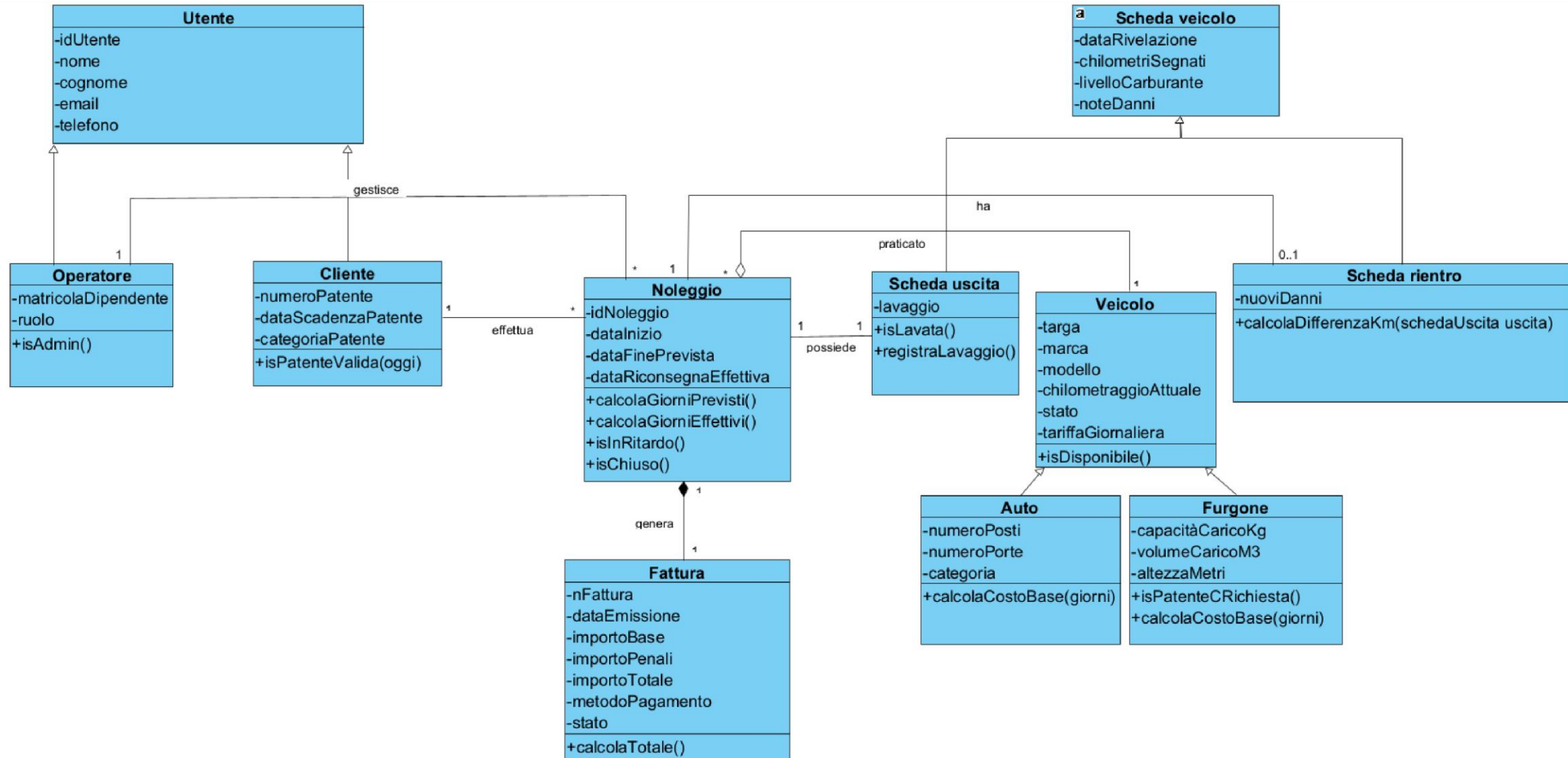
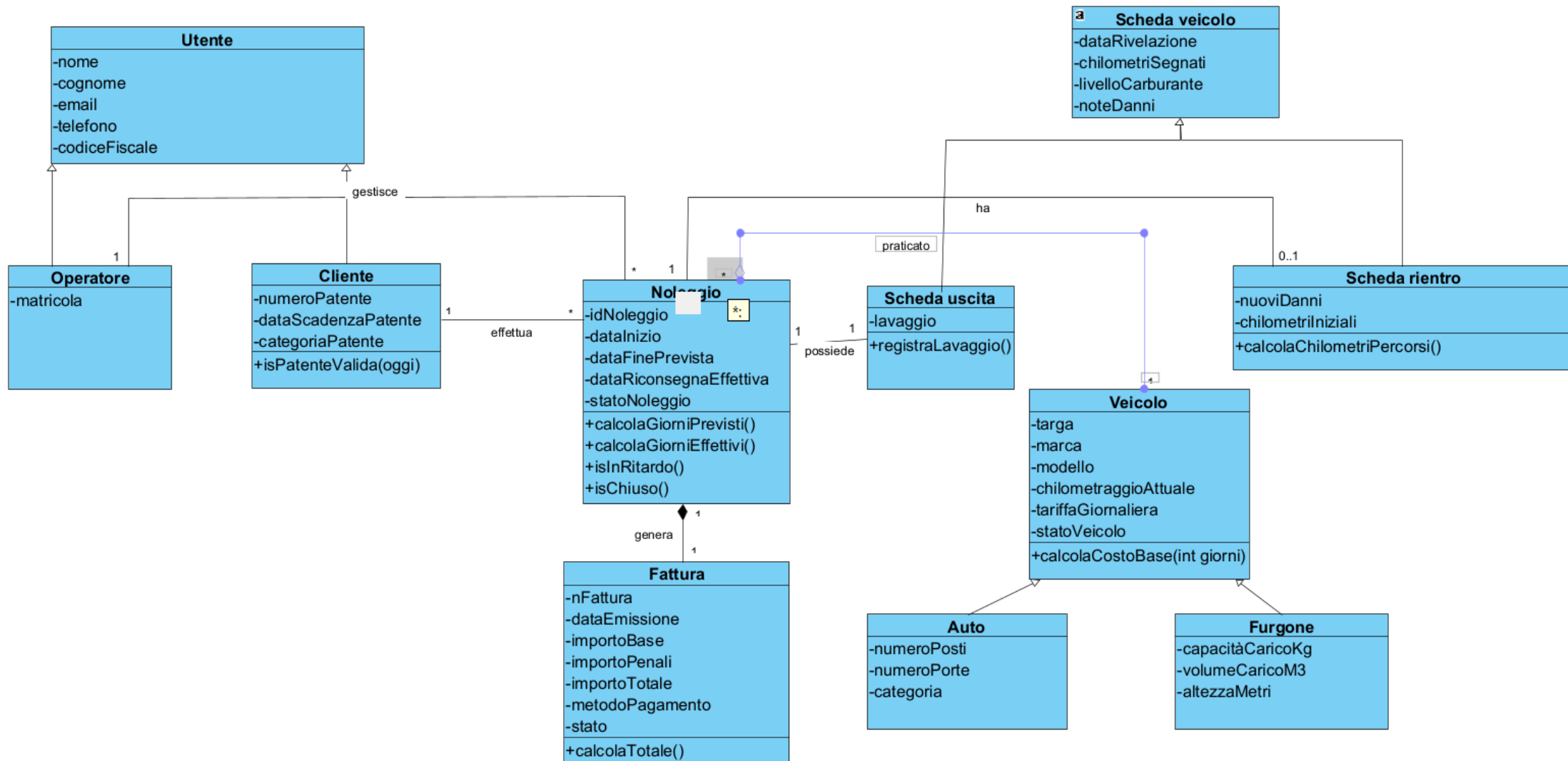


DIAGRAMMA DELLE CLASSI AGGIORNATO



LISTA CAMBIAMENTI

CLASSE UTENTE

- **Cambiamento:** Rimozione dell'attributo idUtente e introduzione di codiceFiscale.

Motivazione: Sostituzione di una chiave primaria con una chiave primaria più intuitiva. Questo approccio previene l'inserimento duplicato della stessa persona fisica nel database e riduce la ridondanza dei dati.

CLASSE OPERATORE

- **Cambiamento:** Rimozione dell'attributo ruolo e del metodo isAdmin().

Motivazione: Semplificazione del dominio. La rimozione è dovuta al fatto che si sta ipotizzando che il programma sia utilizzato da operatori con permessi di accesso ai dati e non direttamente dal cliente. Di conseguenza il ruolo dell'operatore diventa superfluo.

CLASSE NOLEGGIO

- **Cambiamento:** Aggiunta dell'attributo statoNoleggio.

Motivazione: Consente di tracciare esplicitamente il ciclo di vita del noleggio (es. in corso, chiuso) senza doverlo ricalcolare ogni volta confrontando la data attuale con la dataFinePrevista o dataRiconsegnaEffettiva. Semplifica la scrittura della logica di business e l'esecuzione delle query.

CLASSI SCHEDA USCITA/RIENTRO

- **Cambiamento:** Rimozione del metodo `isLavata()` in Scheda uscita.

Motivazione: Rimozione di ridondanza. Avendo l'attributo `lavaggio`, lo stato di pulizia è già accessibile tramite un normale metodo *getter*, rendendo superfluo un metodo specifico.

- **Cambiamento:** Aggiunta dell'attributo `chilometriIniziali` in Scheda rientro e modifica del metodo di calcolo, che ora non richiede più l'oggetto Scheda uscita come parametro.

Motivazione: Trasferendo il dato dei chilometri iniziali nella Scheda rientro al momento in cui viene generata, la classe non ha più bisogno di ricevere in ingresso il parametro della scheda di uscita: ora può calcolare i chilometri percorsi elaborando esclusivamente i propri dati interni.